

TeSP em Inteligência Artificial

Introdução à Inteligência Artificial

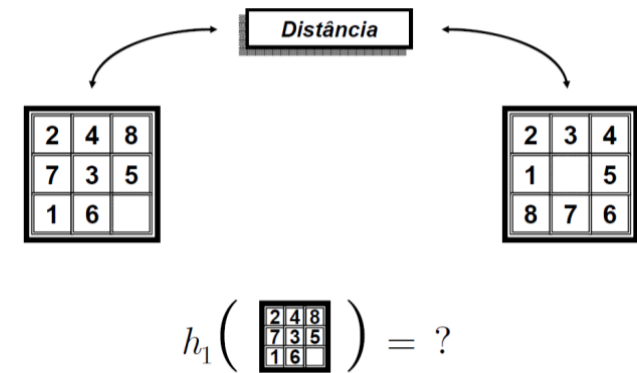
1º ano - 1º semestre – 2025/26

“4. Heurística e Procura Informada”

Professor Doutor **Ricardo Vardasca**
ricardo.vardasca@islasantarem.pt

Indice

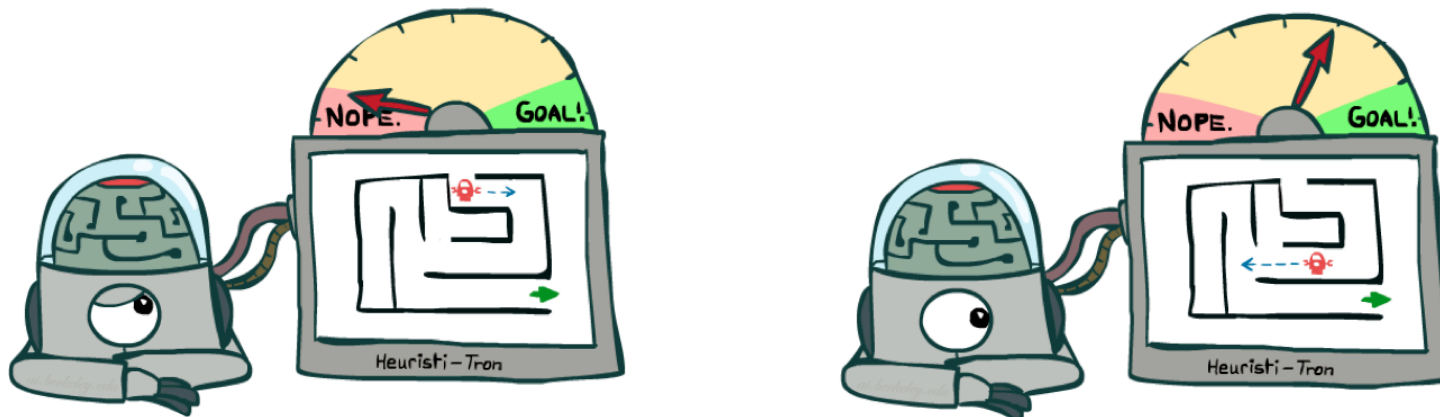
- Função Heurística
- Algoritmos de procura informada
 - Procura Sofrega
 - Procura A*



Função Heurística

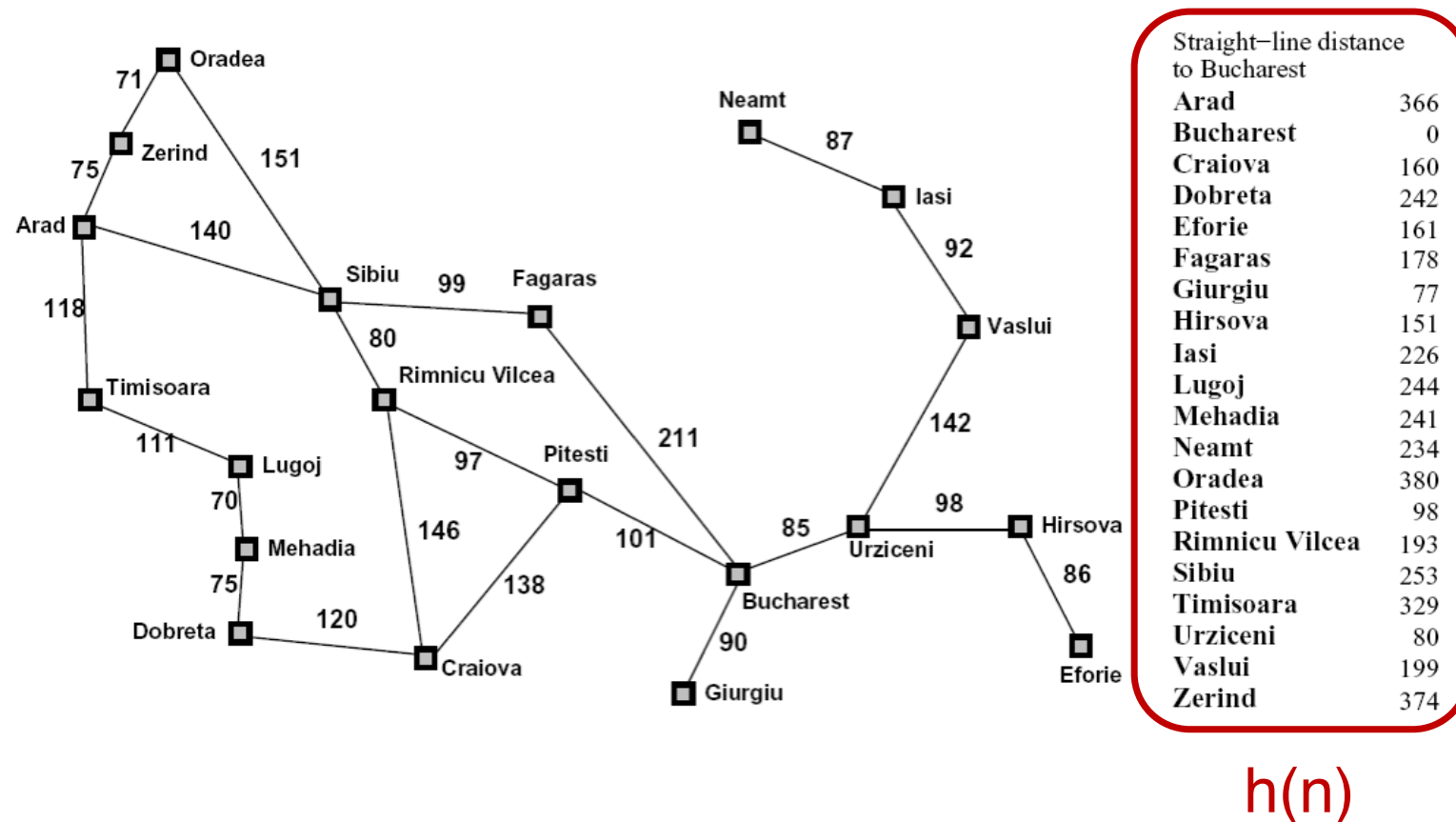
- Uma **função heurística**...

- ...provê uma **estimativa** de quão próximo um estado se encontra de um estado objetivo
- ...é projetada (personalizada) para cada problema de Procura em particular



Exemplo: Função Heurística (Mapa Roménia)

$h(n)$: distância em linha reta desde n até Bucareste



Procura Informada (ou Procura Heurística)

- Utiliza conhecimento específico sobre o problema para encontrar soluções de forma mais eficiente do que a Procura sem informação.
 - Conhecimento específico: além da especificação do problema
- Abordagem geral: **Procura pela Melhor Escolha** (*Best-First Search*)
 - Utiliza uma **função de avaliação**, aplicável a cada nó
 - Expande o nó que tem o valor mais baixo para a função de avaliação (i.e., expande o nó considerado mais promissor)
 - Dependendo da função de avaliação escolhida, a estratégia de Procura muda

Procura Informada (*Informed Search*)

Algoritmos:

- Procura Sofrega (Greedy BFS)
- A^*



Procura pela Melhor Escolha (*BFS*)

- Ideia: usar uma **função de avaliação** $f(n)$ para cada nó.
 - estimativa do quanto aquele nó é desejável
 - Expandir nó mais desejável que ainda não foi expandido
- Implementação: ordenar nós na borda em ordem decrescente do valor da função de avaliação
 - Borda é uma lista de prioridades, ordenada de acordo com o valor de $f(n)$
- Casos especiais:
 - Procura Sofrega (*greedy best-first search, GBFS*)
 - Procura A^* (*A^* search*)

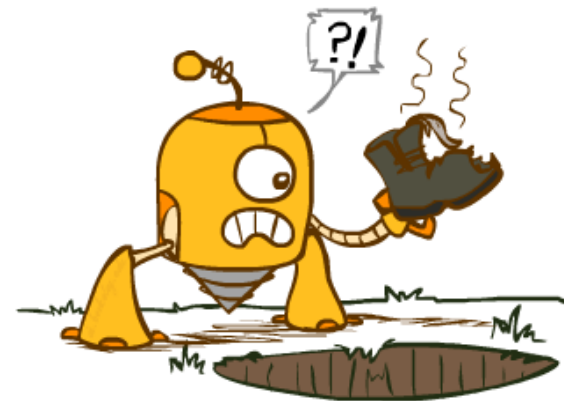
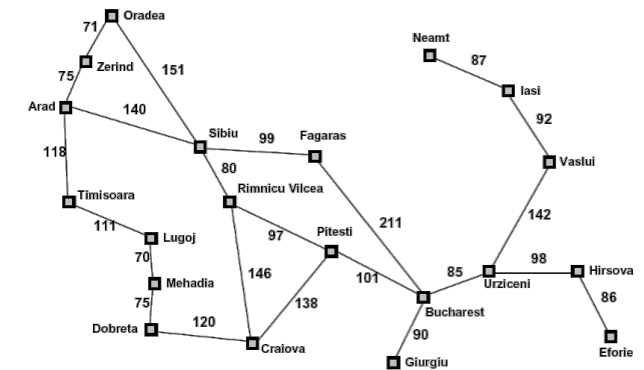
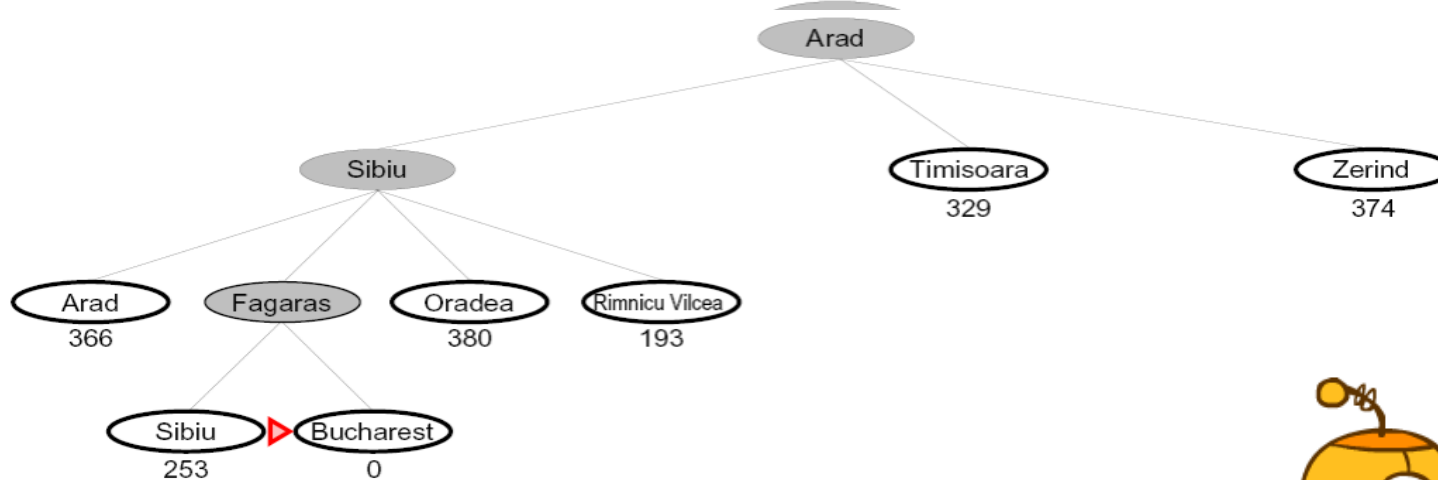
Procura Sofrega (*Greedy BFS*)

- Estratégia: expandir o nó que **parece** ser o mais próximo ao objetivo, de acordo com uma **função heurística**, $h(n)$
 - A esperança é que esse nó leve ao objetivo mais rapidamente
- A função $h(n)$ é uma estimativa do custo desde o nó **n** até o nó objetivo
- Na Procura Sofrega, $f(n) = h(n)$



Exemplo: Roménia

- Procura Sofrega: expandir o nó que *parece* mais próximo ao objetivo

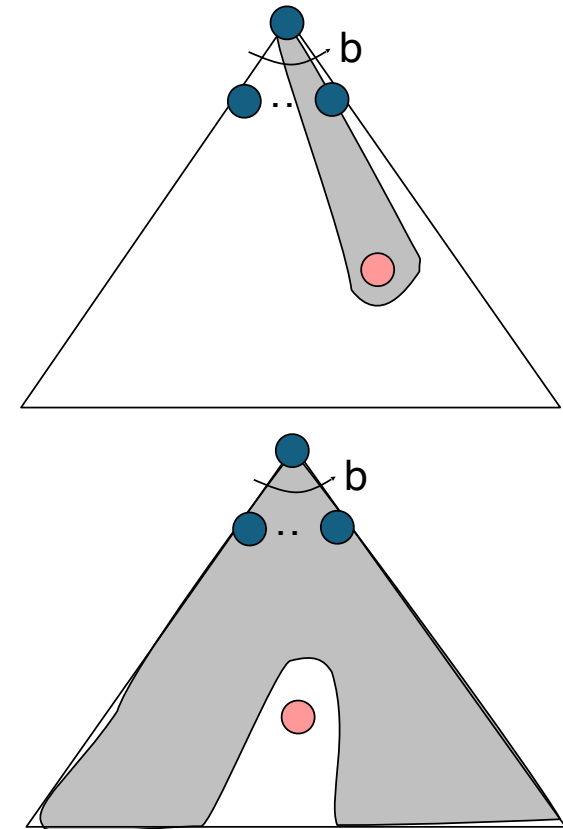


Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

- O que pode acontecer de errado?

Procura Sofrega

- Estratégia: expandir o nó que parece mais próximo a um estado objetivo.
 - Heurística: estimativa da distância ao objetivo mais próximo, para cada estado.
- Um caso comum:
 - GBFS leva diretamente a um objetivo subótimo
- No pior caso: semelhante a um DFS “pior”
- Pois escolhe ação **considerando somente o estado atual**
- Pode haver um plano globalmente melhor que siga algumas opções localmente piores em alguns pontos da árvore de Procura



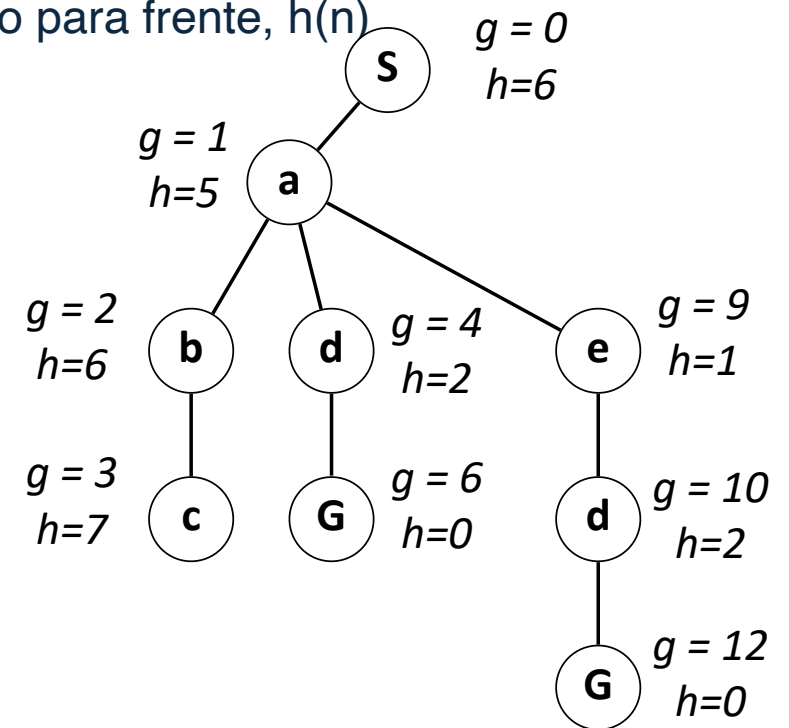
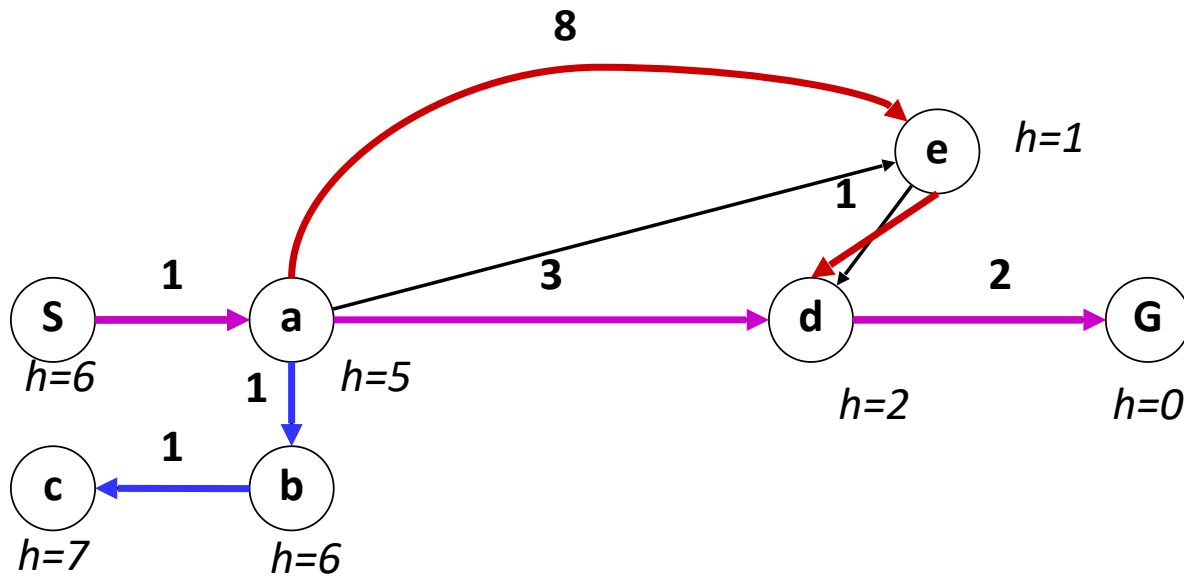
Procura A^*

- Usa uma função de avaliação da forma $f(n) = g(n) + h(n)$
 - $g(n)$ = custo até o momento para alcançar n
 - $h(n)$ = custo estimado de n até o objetivo
 - $f(n)$ = custo total estimado do caminho até o objetivo através de n
- Justificativa: se queremos encontrar a solução menos custosa, uma ação razoável é expandir o nó com o menor valor de $f(n)$
- A^* expande nós de maneira similar à UCS
 - Diferença: UCS usa $g(n)$, enquanto que A^* usa $g(n) + h(n)$



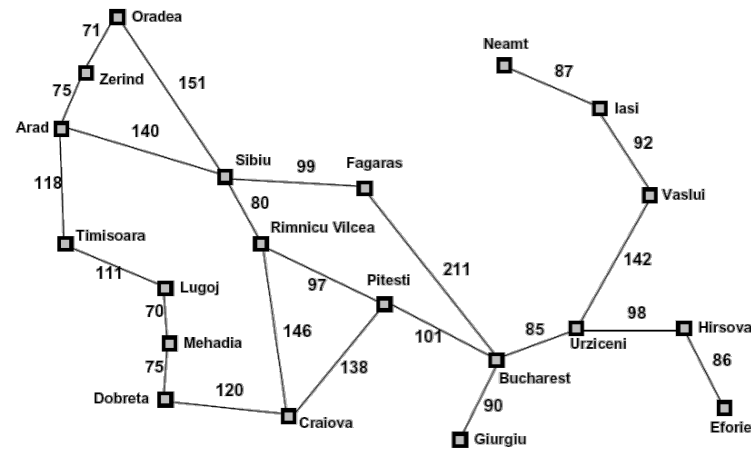
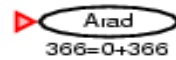
A* combina UCS e Procura Sofrega

- **Procura de Custo Uniforme** ordena por custo de caminho, ou custo para trás, $g(n)$
- **Procura Sofrega** ordena por proximidade estimada, ou custo para frente, $h(n)$



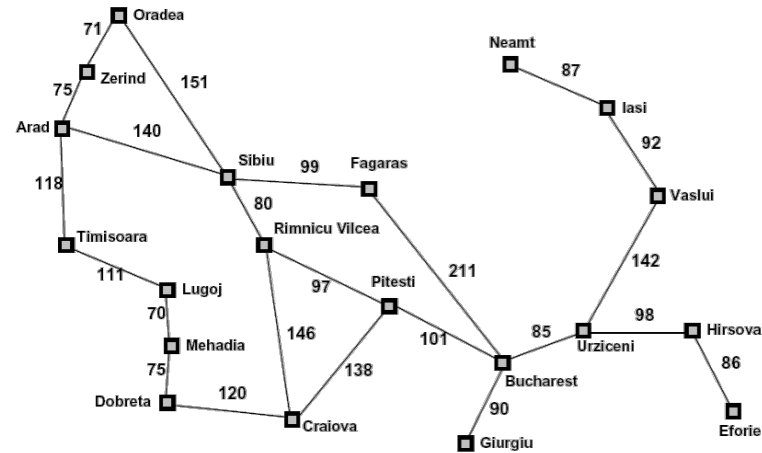
- **Procura A*** ordena pela soma: $f(n) = g(n) + h(n)$

Procura A*: Exemplo (Roménia)



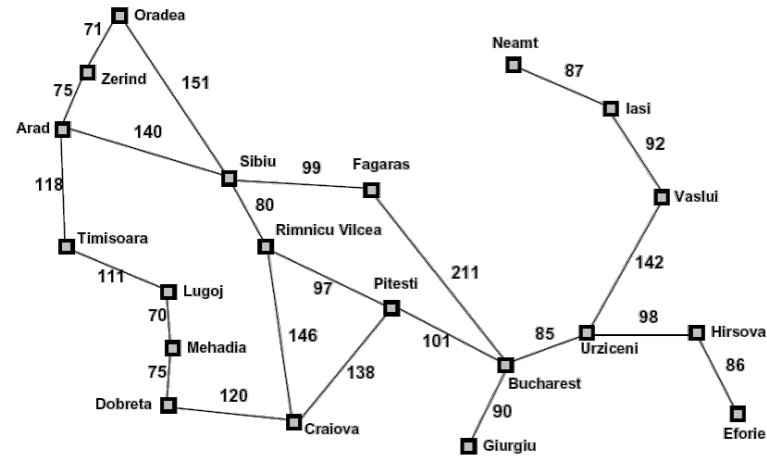
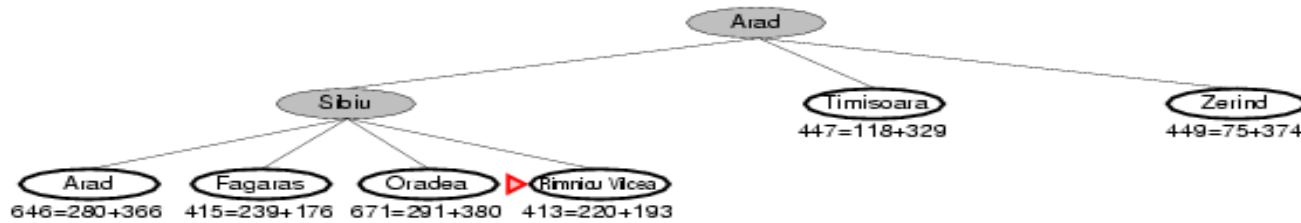
Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Procura A*: Exemplo (Roménia)



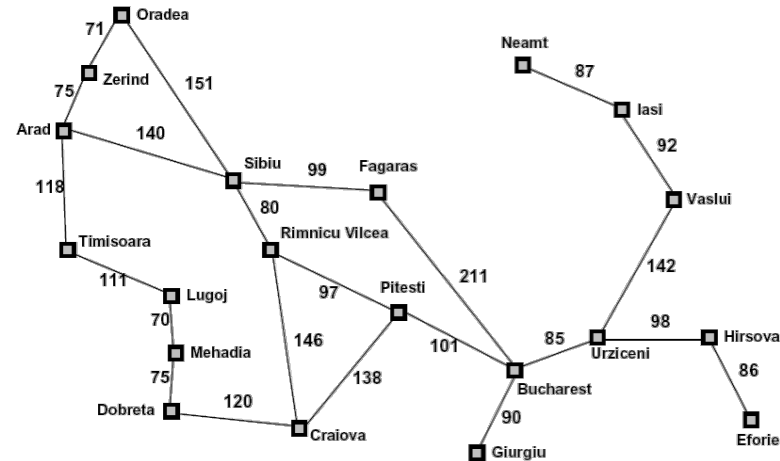
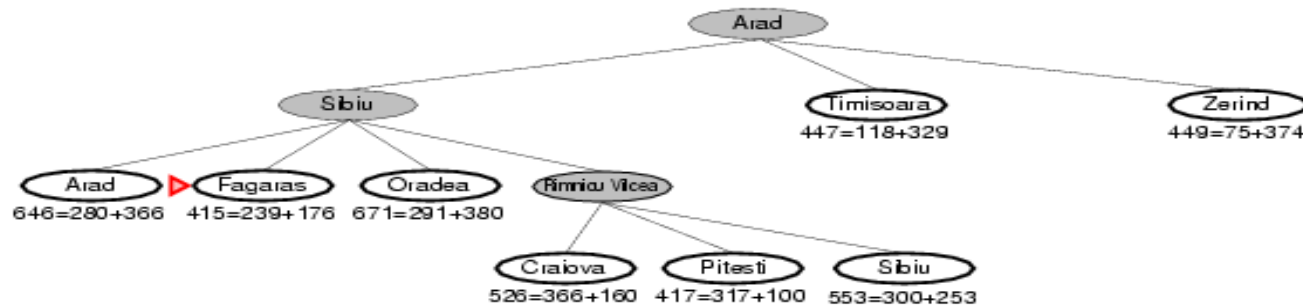
Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Procura A*: Exemplo (Roménia)



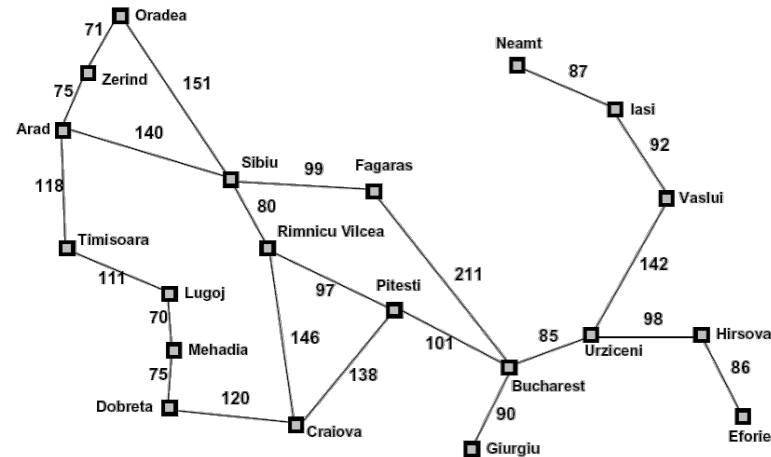
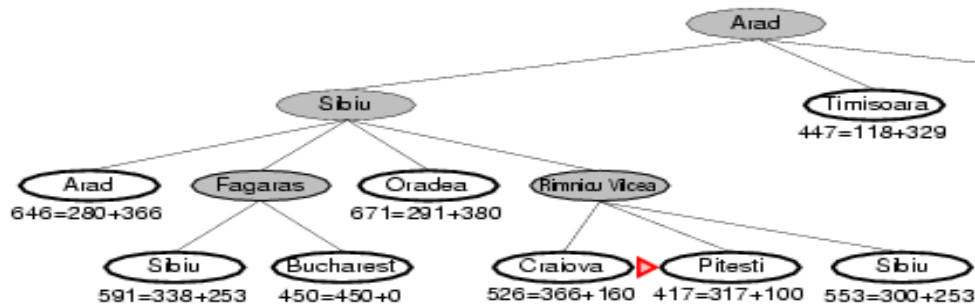
Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Procura A*: Exemplo (Roménia)



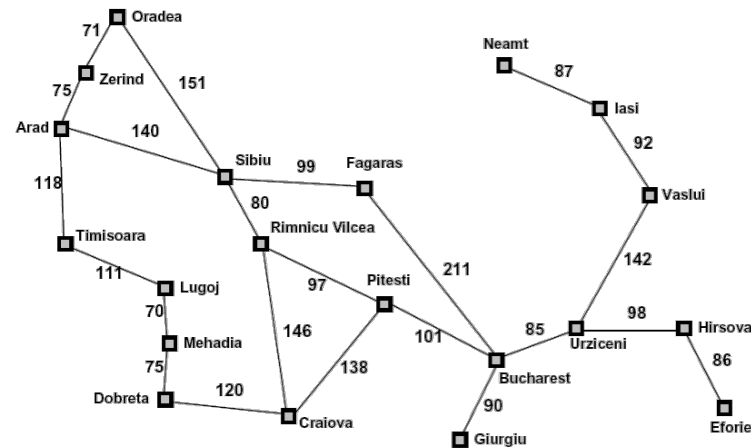
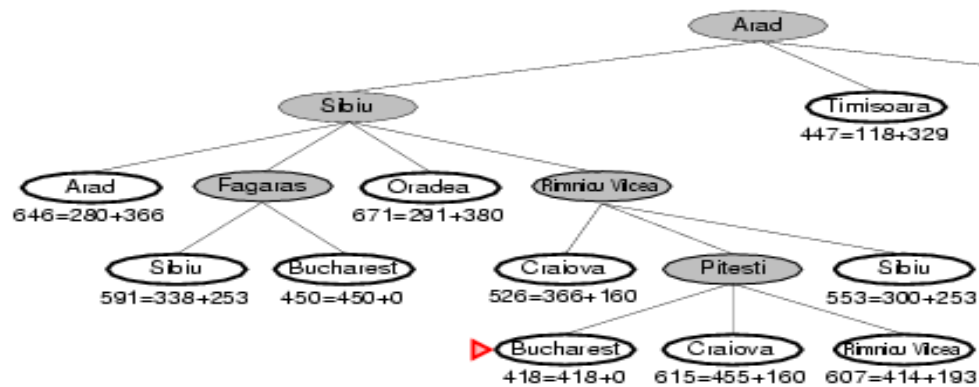
Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Procura A*: Exemplo (Roménia)



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

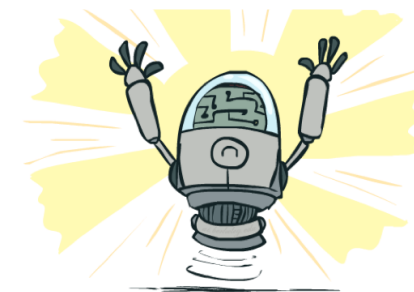
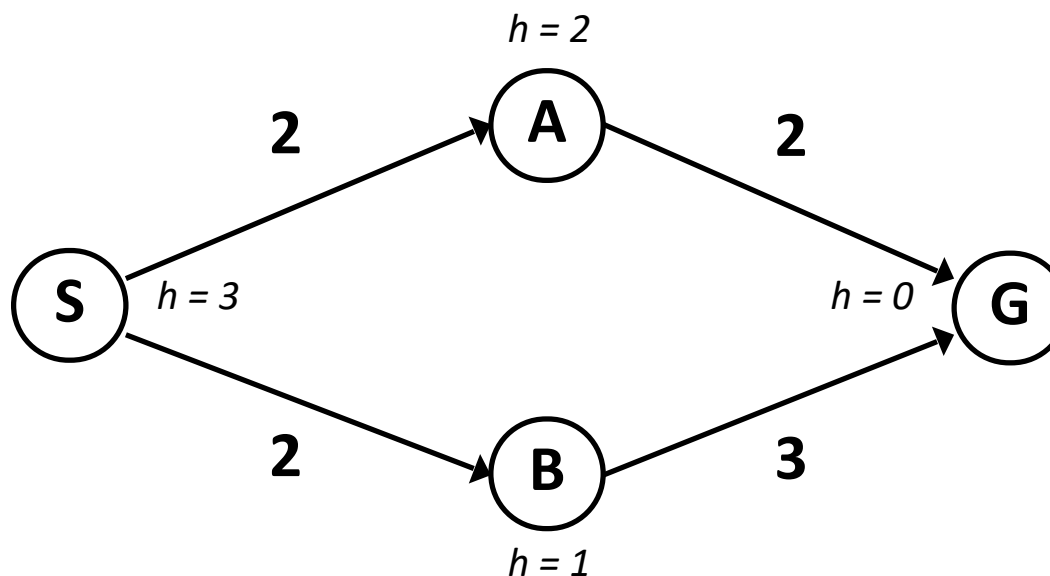
Procura A*: Exemplo (Roménia)



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

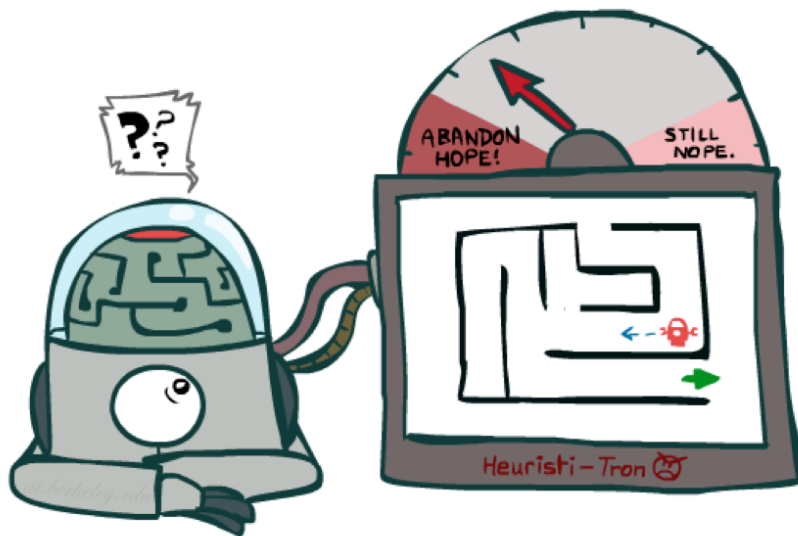
Quando a Procura A* deve terminar?

- Deve terminar quando um nó objetivo é adicionado na fila?

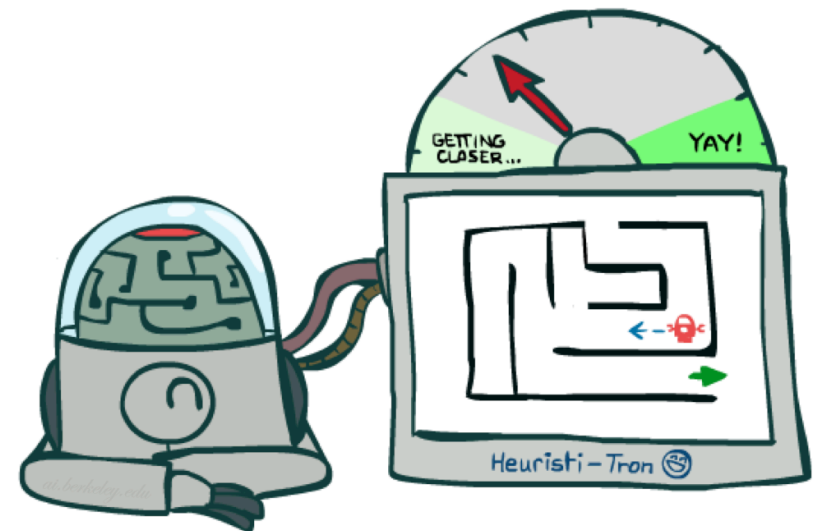


- **Não!** Deve terminar apenas quando um objetivo é retirado da fila

Heurísticas Admissíveis



Heurísticas inadmissíveis (i.e., pessimistas) comprometem (i.e., não garantem) a otimalidade porque “prendem” bons planos na borda



Heurísticas admissíveis (i.e., otimistas) desaceleram planos ruins, e nunca superestimam os custos verdadeiros

- Para o puzzle de 8 peças, considere duas heurísticas:
 - $h_1(n)$ = número de peças fora da posição
 - $h_2(n)$ = distância “Manhattan” total (para cada peça calcular a distância em “quadras” até a sua posição)

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

$h_1(S)$
= ?

8

$$3+1+2+2+2+3+3+2 = 18$$

$h_2(S)$

Comparação

Algoritmo	Completo	Complexidade de tempo	Complexidade de espaço	Ótimo
BFS	Sim (b fin.)	$O(b^d)$	$O(b^d)$	Sim
DFS	Não	$O(b^d)$	$O(bd)$	Não
UCS	Sim ($\geq \epsilon$)	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(b^{\lceil C^*/\epsilon \rceil})$	Sim
Greedy BFS	Sim (b fin.)	$O(b^d)$	$O(b^d)$	Não
A*	Sim	$O(b^d)$	$O(b^d)$	Sim

Métodos de Procura

- **Procura Cega ou Exaustiva:**

- Não sabe qual o melhor nó da fronteira a ser expandido. Apenas distingue o estado objetivo dos não objetivos.

- **Procura Heurística:**

- Estima qual o melhor nó da fronteira a ser expandido com base em funções heurísticas.

- **Procura Local:**

- Operam num único estado e movem-se para a vizinhança deste estado.